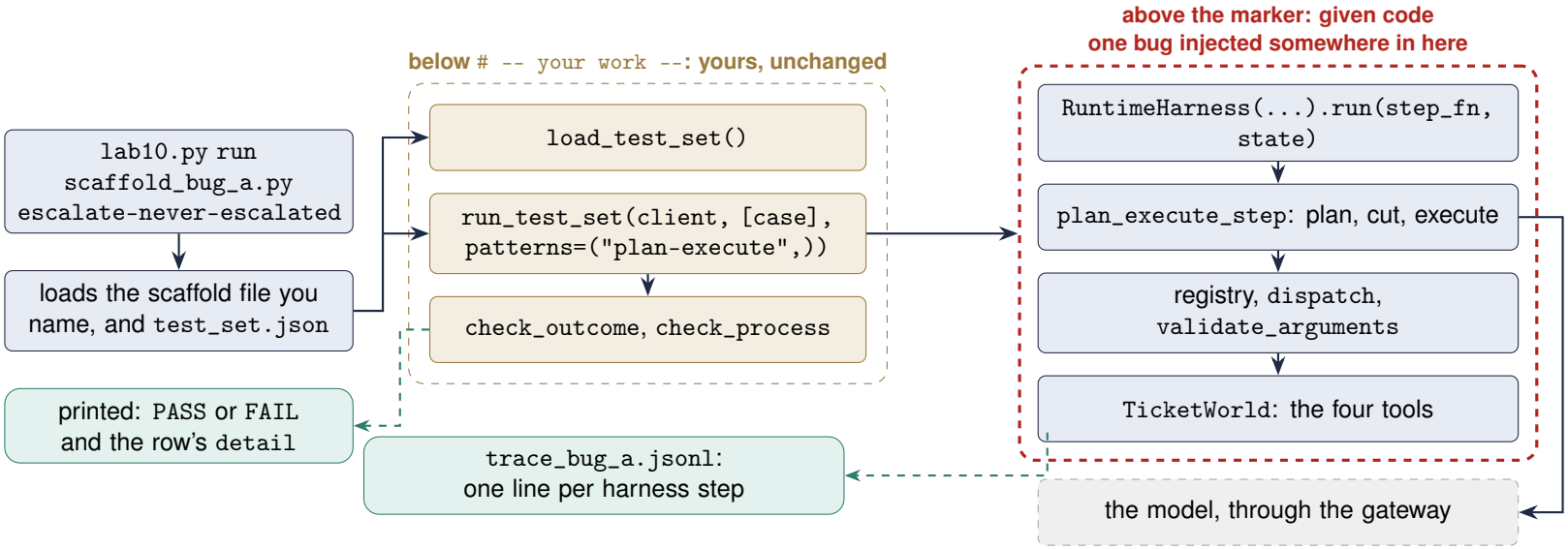
Lab 10 call flow, 1 of 4: who calls whom in `lab10.py run`

One case, one run, your own planner, against the broken file you name. The red dashed box is where a bug can be.



Two broken copies of your own file. *`scaffold_bug_a.py` and `scaffold_bug_b.py` are your Lab 9a `scaffold.py`, each with one bug, in a different place. Your code below the marker is exactly as you left it.*

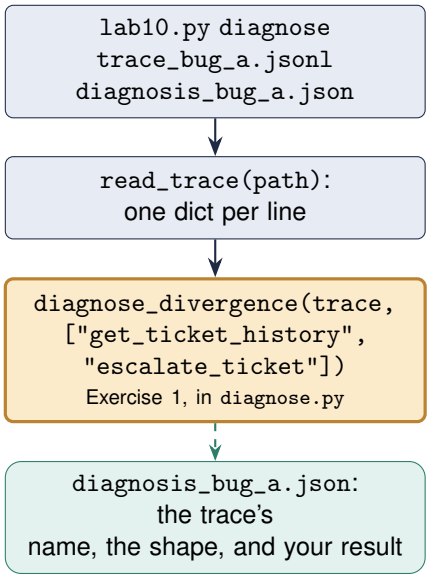
Nothing tells you where. *The run may end `done` or `exhausted`, and no exception points at the line. The trace is the only record of what the agent actually did.*

Your own checks still work. *They are below the marker, untouched, so when a run fails it is your Lab 9a `check_outcome` or `check_process` that says so.*

given code | your Lab 9a code, untouched | you write this | outside your code | data / a file you commit | → calls | - - → returns

Lab 10 call flow, 2 of 4: `lab10.py diagnose` and `diagnose_divergence`

Exercise 1. The trace in, a diagnosis out. An invented example, to show the rules, not either bug.



Step	Tool calls in `output`	Counted?
1	`search_tickets` → `[9, 23]`	yes
2	`check_ticket_status(23)` → `{"error": ...}`	no: an error
2	`{"not_run": ..., "why": ...}`	skip: no `"tool"`
3	`get_ticket_history(23)` → `{"history": ...}`	yes
4	`"planner: done"` (a string)	no tool calls

Invented shape for this example: `["search_tickets", "check_ticket_status"]`.
`search_tickets` matches at step 1. `check_ticket_status` was called, but its result was an error, so it does not count, and nothing later matches it.

```
{"match": false, "missing":
"check_ticket_status", "after_step": 1}
```

In order, with gaps allowed. *Other calls may come between the expected ones; your planner and your neighbour's take different routes. Only the order of the expected tools matters.*

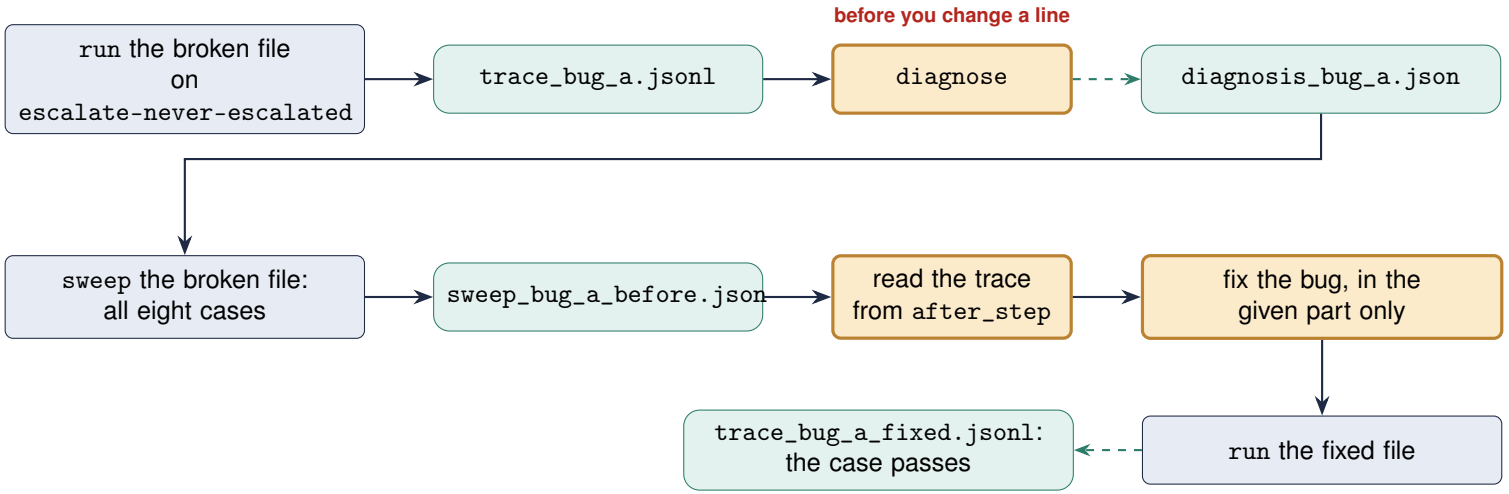
An error is not a success. *A call whose result has an `"error"` key happened, but it did not do its job, so it does not count. Same rule as your Lab 9a `check_process`.*

`after_step` is where the run was last on the path: the step of the last expected tool that did succeed, or `None` if none did. That is where to start reading the trace.

given code | your Lab 9a code, untouched | you write this | outside your code | data / a file you commit | → calls | - - - → returns

Lab 10 call flow, 3 of 4: diagnose first, then fix

Exercises 2 and 3, the same loop once per broken file. Every file you commit comes from one command.



File you commit	Made by	From
`diagnose.py`	you	Exercise 1
`trace_bug_a.jsonl`, `trace_bug_b.jsonl`	`lab10.py run`	the broken file, before the fix
`diagnosis_bug_a.json`, `diagnosis_bug_b.json`	`lab10.py diagnose`	that trace, through your `diagnose_divergence`
`sweep_bug_a_before.json`, `sweep_bug_b_before.json`	`lab10.py sweep`	the broken file, all eight cases
`scaffold_bug_a.py`, `scaffold_bug_b.py`	you	each with its one bug fixed
`trace_bug_a_fixed.jsonl`, `trace_bug_b_fixed.jsonl`	`lab10.py run`	the fixed file
`sweep_bug_a_after.json`, `sweep_bug_b_after.json`	`lab10.py sweep`	the fixed file, all eight cases

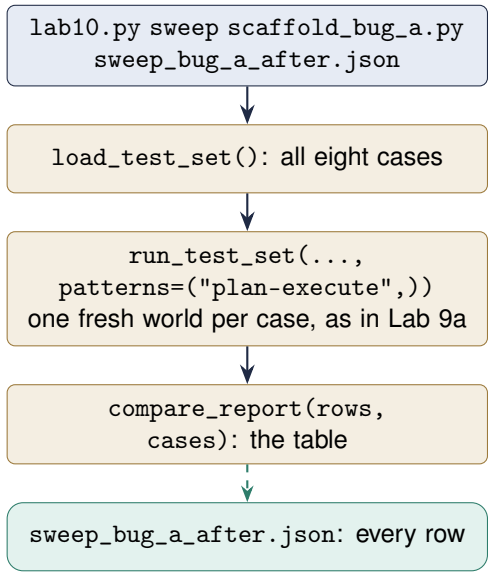
The diagnosis is checked against its trace. *The grader runs a reference `diagnose_divergence` on the trace you committed and compares. A diagnosis written by hand, or from a different run, does not match.*

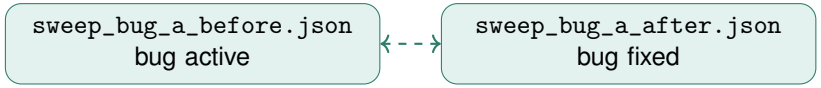
One bug per file, fixed in place. *The two files are never combined, so one failure never hides the other. Your own code below the marker stays as it is.*

given code | your Lab 9a code, untouched | you write this | outside your code | data / a file you commit | → calls | - - → returns

Lab 10 call flow, 4 of 4: `lab10.py sweep`, and where Lab 10 sits

Exercise 4. Your own Lab 9a harness, all eight cases, bugs active against bugs fixed.





Compare them case by case, the way Lab 9b compared two versions: which cases did the bug break, did every one of them recover, and did any case that passed before now fail? A sweep runs eight cases with real model calls, so it takes a few minutes; a run that hits the round limit takes longest.

